CHRONOSMATCH

Python Internship • Matching Engine

Six Core Modules — Step-by-Step Guide

A beginner-friendly guide in easy Hinglish

Is guide ka goal: tumhe ChronosMatch ke six modules samjhana, unka role batana, aur Matching Engine wale kaam par focus karwana.

Project context: ChronosMatch ek simulated exchange engine hai. Ye sample buy/sell orders ko manage karta hai, compatible orders match karta hai, trades record karta hai aur performance measure karta hai. Ye real-money trading system nahi hai.

Tumhara assigned role: Matching Engine — isliye Module 1 aur Module 2 ko achchhe se samjho. Baaki modules ka flow bhi samajhna zaroori hai, kyunki engine API, storage aur tests ke saath integrate hoga.

Project ko ek nazar mein samjho

User ya API order bhejta hai → validation hoti hai → Matching Engine opposite side ke Order Book ko check karta hai → compatible orders par trade banta hai → order/trade data store hota hai → dashboard par result dikhta hai.

1. Order Book

Simple meaning: Pending buy aur sell orders ko organize karke rakhna.

Is module ke main kaam

• Har symbol (jaise ABC) ke liye alag orders maintain karna.

• Buy side ko bids aur sell side ko asks kaha jaata hai.

• Best bid = sabse high buy price; best ask = sabse low sell price.

• Same price par pehle aaya order pehle process hota hai (FIFO).

Example: Example: Buy orders ₹100 aur ₹102 par hain. Best bid ₹102 hoga. Sell orders ₹105 aur ₹103 par hain. Best ask ₹103 hoga.

Tumhare role se connection: Matching Engine ko Order Book se best eligible opposite order milta hai. Tumhare role mein order-book interface samajhna important hai.

2. Matching Engine

Simple meaning: Project ka core logic: compatible buy aur sell orders ko match karna.

Is module ke main kaam

• Buy limit price >= sell limit price ho, tab trade possible hai.

• Better price ko priority milti hai; same price par older order ko priority milti hai.

• Trade price prototype mein resting order ka price hota hai.

• Partial fill mein dono orders ki remaining quantity update hoti hai.

• Alag symbols ke orders kabhi match nahi hone chahiye.

Example: Buyer: 10 shares @ ₹100. Seller: 6 shares @ ₹100. Trade quantity 6 hogi; buyer ke 4 shares remaining rahenge.

Tumhare role se connection: Tumhara primary ownership area. Pehle single order pair ka logic, phir multiple orders, price-time priority, partial fills aur cancellation implement karo.

3. API — FastAPI + Pydantic

Simple meaning: Engine ko HTTP requests ke through use karne ka interface.

Is module ke main kaam

• POST /orders — naya order submit karna.

• GET /orders/{order_id} — order status dekhna.

• DELETE /orders/{order_id} — eligible open quantity cancel karna.

• GET /orderbook/{symbol} — order-book snapshot lena.

• GET /trades, /stats, /health — trades, statistics aur health check.

Example: Dashboard ya API client POST /orders bhejta hai. API input validate karke engine ko order deti hai aur response return karti hai.

Tumhare role se connection: API code ko matching logic ke andar mix mat karo. Engine ko independently testable rakho.

4. Database / Storage — SQLite

Simple meaning: Orders aur trades ka history save karna.

Is module ke main kaam

• Order aur trade records persist karna.

• Trade ID, symbol, price, quantity, buyer/seller order IDs aur timestamp store karna.

• History ko restart ke baad retrieve kar pana storage design par depend karega.

• Database transaction boundaries consistent records ke liye important hain.

Example: Trade execute hone par uska record database mein save hota hai, jisse baad mein trade history dekhi ja sakti hai.

Tumhare role se connection: Engine result/events generate kare; storage layer unhe persist kare. Database code ko matching rules se alag rakho.

5. Testing & Performance / Benchmark

Simple meaning: Correctness prove karna aur speed/memory ko repeatable tareeke se measure karna.

Is module ke main kaam

• pytest se unit, integration aur regression tests likhna.

• Full fill, partial fill, no match, FIFO, cancellation aur symbol isolation test karna.

• time.perf_counter_ns() se timing measure karna.

• cProfile se CPU hot spots aur tracemalloc se allocations inspect karna.

• Throughput, median, p95/p99 latency, memory aur errors report karna.

Example: Same workload aur same machine par baseline aur optimized version compare karo. Actual measurements hi report karo.

Tumhare role se connection: Optimization se pehle correct baseline banao. Tests optimization ke baad bhi pass hone chahiye.

6. Dashboard — Streamlit

Simple meaning: System ka result simple visual screen par dikhana.

Is module ke main kaam

- Order-entry form se sample order submit karna.
- Order book aur active order status dekhna.
- Trade history inspect karna.
- Basic statistics aur benchmark information display karna.

Example: Reviewer dashboard par order submit karta hai aur dekh sakta hai ki trade bana ya order pending raha.

Tumhare role se connection: Dashboard optional presentation layer hai. Core Matching Engine ko dashboard ke bina bhi run aur test kar pana chahiye.

Matching Engine: pehla working concept

Neeche ka code sirf learning example hai. Ye ek buyer aur ek seller ko match karta hai; complete project engine nahi hai.

```
def match_orders(buy_order, sell_order):
# Buyer ka price seller ke price se kam hai?
if buy_order["price"] < sell_order["price"]:
return "No trade: prices do not match"

# Dono mein se chhoti quantity execute hogi
traded_quantity = min(
buy_order["quantity"],
sell_order["quantity"]
)

# Remaining quantity update karo
buy_order["quantity"] -= traded_quantity
sell_order["quantity"] -= traded_quantity

return {
"trade_price": sell_order["price"],
"trade_quantity": traded_quantity,
"buyer_remaining": buy_order["quantity"],
"seller_remaining": sell_order["quantity"]
}


buyer = {"price": 100, "quantity": 10}
seller = {"price": 100, "quantity": 6}

result = match_orders(buyer, seller)
print(result)
```

Expected result

```
{'trade_price': 100, 'trade_quantity': 6, 'buyer_remaining': 4, 'seller_remaining': 0}
```

Important: Is starter code mein abhi order ID, symbol check, order status, FIFO, order book, cancellation aur validation nahi hain. In features ko step-by-step add karna hai.

Tumhara 4-week implementation roadmap

Week	Focus	Expected progress
1	Requirements, models, basic order book, test setup	Repository runs; basic book tests pass
2	Matching, partial fills, cancellation, order lifecycle	Order submit se trade record tak demo
3	API/storage integration, tests, baseline benchmark	Integration reliable; benchmark method recorded
4	Bug fixes, optimization evaluation, docs and demo	Final tests, report, presentation and demo

Suggested project file structure

```
ChronosMatch/
app/
engine/
models.py
order_book.py
matching_engine.py
tests/
test_order_book.py
test_matching_engine.py
benchmarks/
benchmark_engine.py
README.md
```

Ye target structure hai. Files ko feature banate waqt add karo; sirf diagram match karne ke liye empty files mat banao.

Matching Engine checklist

Har item complete hone par tick lagao. Pehle correctness, phir performance.

☐ Order model mein ID, symbol, side, price, quantity aur status hai.

☐ Invalid price, quantity aur side reject hote hain.

☐ Different symbols ke orders match nahi hote.

☐ Buy price >= sell price condition sahi hai.

☐ Best price priority follow hoti hai.

☐ Same price par FIFO / time priority follow hoti hai.

☐ Full fill aur partial fill sahi update hote hain.

☐ Remaining quantity non-negative rehti hai.

☐ Trade record mein IDs, price, quantity aur timestamp hote hain.

☐ Eligible open order cancellation kaam karti hai.

☐ Tests pass hote hain aur benchmark repeatable hai.

Useful Python topics to revise

Topic	ChronosMatch mein use
Functions	Matching logic ko reusable banana
Classes / objects	Order, Trade aur Engine models
Lists / dictionaries	Orders aur lookup data rakhna
deque / FIFO	Same price par time priority
Enums	BUY, SELL aur order statuses
pytest	Automatic tests likhna
Git / GitHub	Branch, commit aur pull request workflow

Final advice: Tumhara primary focus Matching Engine hai. Pehle ek order pair, phir multiple orders, phir price-time priority, partial fills aur cancellation. Har feature ke saath test likho. Zero-copy ya advanced optimization tabhi karo jab baseline correct ho aur profiling se bottleneck mile.

Source basis: ChronosMatch Project Blueprint & Team Execution Document supplied in this chat. The six modules are explained using its proposed architecture and scope. Technology and design choices are proposals to confirm with the project mentor.